

day10

服务端解决多线程并发安全问题

为了让能叫消息转发给所有客户端，我们 在Server上添加了一个集合类型的属性allOut,并且共所有线程ClientHandler使用，这时对集合的操作要考虑并发安全问题，还要考虑对集合的不同操作之间的互斥问题。因此，对allOut集合的添加元素，删除元素和遍历操作要进行互斥。

最终代码:

```
package socket;

import java.io.*;
import java.net.ServerSocket;
import java.nio.charset.StandardCharsets;
import java.net.Socket;

/**
 * 聊天室服务端
 */
public class Server {
    /**
     * 运行在服务端的ServerSocket主要完成两个工作：
     * 1: 向服务端操作系统申请服务端口，客户端就是通过这个端口与ServerSocket建立链接
     * 2: 监听端口，一旦一个客户端建立链接，会立即返回一个Socket。通过这个Socket
     * 就可以和该客户端交互了
     *
     * 我们可以把ServerSocket想象成某客服的"总机"。用户打电话到总机，总机分配一个
     * 电话使得服务端与你沟通。
     */
    private ServerSocket serverSocket;
    /**
     * 存放所有客户端输出流，用于广播消息
     */
    private PrintWriter[] allOut = {};

    /**
     * 服务端构造方法，用来初始化
     */
    public Server(){
        try {
            System.out.println("正在启动服务端...");
            /**
             * 实例化ServerSocket时要指定服务端口，该端口不能与操作系统其他
             * 应用程序占用的端口相同，否则会抛出异常：
             * java.net.BindException:address already in use
             *
             * 端口是一个数字，取值范围:0-65535之间。
             * 6000之前的端口不要使用，密集绑定系统应用和流行应用程序。
             */
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

        */
        serverSocket = new ServerSocket(8088);
        System.out.println("服务端启动完毕!");
    } catch (IOException e) {
        e.printStackTrace();
    }
}

/**
 * 服务端开始工作的方法
 */
public void start(){
    try {
        while(true) {
            System.out.println("等待客户端链接...");
            /*
                ServerSocket提供了接受客户端链接的方法:
                Socket accept()
                这个方法是一个阻塞方法, 调用后方法"卡住", 此时开始等待客户端
                的连接, 直到一个客户端链接, 此时该方法会立即返回一个Socket实例
                通过这个Socket就可以与客户端进行交互了。

                可以理解为此操作是接电话, 电话没响时就一直等。
            */
            Socket socket = serverSocket.accept();
            System.out.println("一个客户端链接了! ");
            //启动一个线程与该客户端交互
            ClientHandler clientHandler = new ClientHandler(socket);
            Thread t = new Thread(clientHandler);
            t.start();

        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    Server server = new Server();
    server.start();
}

/**
 * 定义线程任务
 * 目的是让一个线程完成与特定客户端的交互工作
 */
private class ClientHandler implements Runnable{
    private Socket socket;
    private String host;//记录客户端的IP地址信息

    public ClientHandler(Socket socket){
        this.socket = socket;
        //通过socket获取远端计算机地址信息
    }
}

```

```

        host = socket.getInetAddress().getHostAddress();
    }
    public void run(){
        PrintWriter pw = null;
        try{
            /*
             Socket提供的方法:
             InputStream getInputStream()
             获取的字节输入流读取的是对方计算机发送过来的字节
            */
            InputStream in = socket.getInputStream();
            InputStreamReader isr = new InputStreamReader(in,
StandardCharsets.UTF_8);
            BufferedReader br = new BufferedReader(isr);

            OutputStream out = socket.getOutputStream();
            OutputStreamWriter osw = new
OutputStreamWriter(out,StandardCharsets.UTF_8);
            BufferedWriter bw = new BufferedWriter(osw);
            pw = new PrintWriter(bw,true);

            //将该输出流存入allout中
            synchronized (allout) {
                allout.add(pw);
            }
            //通知所有客户端该用户上线了
            System.out.println(host + "上线了,当前在线人数:"+allout.length);

            String message = null;
            while ((message = br.readLine()) != null) {
                System.out.println(host + "说:" + message);
                //将消息回复给所有客户端
                synchronized (allout) {
                    for (PrintWriter o : allout) {
                        allout[i].println(host + "说:" + message);
                    }
                }
            }
        } catch (IOException e){
            e.printStackTrace();
        } finally{
            //处理客户端断开链接的操作
            //将当前客户端的输出流从allout中删除
            synchronized (allout) {
                allout.remove(pw);
            }
            System.out.println(host+"下线了, 当前在线人数:"+allout.length);
            try {
                socket.close();//与客户端断开链接
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```
    }  
    }  
}
```

Map 查找表

Map体现的结构是一个多行两列的表格,其中左列称为key,右列称为value.

- Map总是成对保存数据,并且总是根据key获取对应的value.因此我们可以将查询的条件作为key查询对应的结果作为value保存到Map中.
- Map有一个要求:key不允许重复(equals比较的结果)

java.util.Map接口,是所有Map的顶级接口,规定了Map的相关功能.

常用实现类:

- java.util.HashMap:称为散列表,使用散列算法实现的Map,当今查询速度最快的数据结构.
- java.util.TreeMap:使用二叉树实现的Map

```
package map;  
  
import java.util.HashMap;  
import java.util.Map;  
  
/**  
 * java.util.Map接口 查找表  
 * Map体现的结构像是一个多行两列的表格, 其中左列称为key, 右列称为value  
 * Map总是成对儿(key-value键值对)保存数据, 并且总是根据key获取其对应的value  
 *  
 * 常用实现类:  
 * java.util.HashMap:称为散列表, 使用散列算法实现的Map, 当今查询速度最快的  
 * 数据结构。  
 */  
public class MapDemo {  
    public static void main(String[] args) {  
        Map<String,Integer> map = new HashMap<>();  
        /*  
         * put(K k,V v)  
         * 将给定的键值对儿存入Map  
         * Map有一个要求, 即: Key不允许重复(Key的equals比较)  
         * 因此如果使用重复的key存入value, 则是替换value操作, 此时put方法  
         * 的返回值就是被替换的value。否则返回值为null。  
         */  
        /*  
         * 注意, 如果value的类型是包装类, 切记不要用基本类型接收返回值,  
         * 避免因为自动拆箱导致的空指针  
         */  
        Integer value = map.put("语文",99);  
        System.out.println(value);//null  
        map.put("数学",98);  
        map.put("英语",97);  
        map.put("物理",96);  
    }  
}
```

```

map.put("化学",98);
System.out.println(map);

value = map.put("物理",66);
System.out.println(value);//96,物理被替换的值
System.out.println(map);

/*
    V get(Object key)
    根据给定的key获取对应的value。若给定的key不存在则返回值为null
*/
value = map.get("语文");
System.out.println("语文:"+value);

value = map.get("体育");
System.out.println("体育:"+value);//null

int size = map.size();
System.out.println("size:"+size);
/*
    V remove(Object key)
    删除给定的key所对应的键值对，返回值为这个key对应的value
*/
value = map.remove("语文");
System.out.println(map);
System.out.println(value);

/*
    boolean containsKey(Object key)
    判断当前Map是否包含给定的key
    boolean containsValue(Object value)
    判断当前Map是否包含给定的value
*/
boolean ck = map.containsKey("英语");
System.out.println("包含key:"+ck);
boolean cv = map.containsValue(97);
System.out.println("包含value:"+cv);
}
}

```

Map的遍历

Map支持三种遍历方式:

- 遍历所有的key
- 遍历所有的键值对
- 遍历所有的value(相对不常用)

```
package map;
```

```

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

/**
 * Map的遍历
 * Map提供了三种遍历方式
 * 1:遍历所有的key
 * 2:遍历每一组键值对
 * 3:遍历所有的value(不常用)
 */
public class MapDemo2 {
    public static void main(String[] args) {
        Map<String,Integer> map = new HashMap<>();
        map.put("语文",99);
        map.put("数学",98);
        map.put("英语",97);
        map.put("物理",96);
        map.put("化学",98);
        System.out.println(map);

        /**
         * 遍历所有的key
         * Set keySet()
         * 将当前Map中所有的key以一个Set集合形式返回。遍历该集合就等同于
         * 遍历了所有的key
         */
        Set<String> keySet = map.keySet();
        for(String key : keySet){
            System.out.println("key:"+key);
        }

        /**
         * 遍历每一组键值对
         * Set<Entry> entrySet()
         * 将当前Map中每一组键值对以一个Entry实例形式存入Set集合后返回。
         * java.util.Map.Entry
         * Entry的每一个实例用于表示Map中的一组键值对，其中有两个常用方法：
         * getKey()和getValue()
         */
        Set<Map.Entry<String,Integer>> entrySet = map.entrySet();
        for(Map.Entry<String,Integer> e : entrySet){
            String key = e.getKey();
            Integer value = e.getValue();
            System.out.println(key+": "+value);
        }

        /**
         * JDK8之后集合框架支持了使用lambda表达式遍历。因此Map和Collection都
         * 提供了foreach方法，通过lambda表达式遍历元素。
         */
        map.forEach(
            (k,v)-> System.out.println(k+": "+v)
        );
    }
}

```

```

        /*
        遍历所有的value
        Collection values()
        将当前Map中所有的value以一个集合形式返回
        */
        Collection<Integer> values = map.values();
        //      for(Integer i : values){
        //          System.out.println("value:"+i);
        //      }
        /*
        集合在使用foreach遍历时并不要求过程中不能通过集合的方法增删元素。
        而之前的迭代器则有此要求，否则可能在遍历过程中抛出异常。
        */
        values.forEach(
            i -> System.out.println("value:"+i)
        );
    }
}

```

总结

java.util.Map 查找表

特点:体现的结构是一个多行两列的表格，其中左列称为key，右列称为value。

Map中的key不允许重复。判定重复的标准是根据key的equals方法判定的。

常用的实现类:java.util.HashMap 散列表

常用方法:

V put(K k,V v):向Map中添加一组键值对,使用重复的key存入新的value时，那么就是替换value操作。此时put方法返回值为被替换的value。否则返回值为null。

V get(K k):根据给定的key获取对应的value。如果给定的key不存在则返回值为null

V remove(K k):根据给定key从Map中删除对应的键值对，返回值为该key对应的value。

int size():返回当前Map中的元素个数

void clear():清空Map

boolean containsKey(Object key):判断当前的Map是否包含给定的key

boolean containsValue(Object value):判断当前Map是否包含给定的value

Set keySet():遍历key使用的方法，将当前Map中所有的key以一个Set集合形式返回

Set entrySet():遍历每一组键值对的方法，将当前Map中每一组键值对(Entry实例)以一个Set集合形式返回

Collection values():遍历所有value使用的方法，将当前Map中所有的value以一个集合形式返回

forEach():基于lambda表达式遍历Map